T-20 CRICKETMATCHSIMULATION

A Simulation-Based Study of Concurrency, Synchronization, and Deadlock Handling

Team Members:

Rathod Anshul ram – 24114074
Ishpreet Kaur -- 24114044

ABSTRACT

This project presents a multithreaded simulation of a cricket match implemented using C++ and POSIX threads (pthreads). The primary objective is to model the dynamic interactions between different entities such as batsmen, bowlers, and fielders while maintaining synchronization in a concurrent environment. Cricket, being a real-time and event-driven game, serves as an effective case study for demonstrating concepts of multithreading, scheduling, and resource sharing.

In this simulation, each player is represented as an independent thread, allowing multiple actions to occur concurrently. A scheduler module is used to coordinate the sequence of events such as ball delivery, shot selection, and score updates. Shared resources like the scoreboard are protected using mutex locks to prevent race conditions and ensure data consistency. Additionally, mechanisms for handling synchronization issues and preventing deadlocks are incorporated.

The system generates realistic match outputs, including runs scored, wickets taken, and over-by-over progress. Visualization of results is achieved through graphs representing match dynamics. This project demonstrates practical applications of operating system concepts such as concurrency control, thread management, and synchronization in a real-world scenario.

INTRODUCTION

Simulation plays a crucial role in understanding and analyzing real-world systems where multiple events occur simultaneously. It allows complex processes to be modeled in a controlled environment, making it easier to study behavior, performance, and interactions. One such dynamic system is a cricket match, which involves multiple players, continuous events, and real-time decision-making.

In a typical cricket match, several activities occur concurrently, such as bowling, batting, fielding, and score updating. These actions are interdependent and require proper coordination to ensure consistency and realism. Modeling such a system requires the use of concurrency concepts, where multiple entities operate simultaneously while sharing common resources. Without proper synchronization, issues such as race conditions and inconsistent data can arise.

This project focuses on designing and implementing a multithreaded cricket match simulation using C++ and POSIX threads (pthreads). Each player in the simulation is represented as a separate thread, enabling concurrent execution of actions. A scheduling mechanism is used to control the flow of the game, while synchronization techniques such as mutex locks are employed to manage shared resources like the scoreboard.

The project demonstrates how operating system concepts such as multithreading, synchronization, and deadlock handling can be applied to simulate a real-world scenario effectively.

OBJECTIVES

The primary objectives of this project are as follows:

- To design and develop a simulation of a cricket match that models real-time interactions between players such as batsmen, bowlers, and fielders.
- To implement multithreading using C++ and POSIX threads (pthreads) in order to simulate concurrent execution of different entities in the system.
- To demonstrate the use of synchronization mechanisms such as mutex locks for managing shared resources and preventing race conditions.
- To develop a scheduling mechanism that controls the sequence of events in the match, including ball delivery, shot execution, and score updates.
- To implement safe handling of shared data structures such as the scoreboard to ensure consistency of match results.
- To analyze and prevent potential deadlock situations that may arise due to improper resource allocation and thread interaction.
- To generate meaningful outputs and performance metrics such as runs scored, wickets taken, and over-by-over progression of the match.
- To visualize the simulation results using graphs and charts for better understanding of match dynamics.
- To apply theoretical concepts of operating systems, such as concurrency, synchronization, and thread management, in a practical real-world scenario.

SYSTEM DESIGN

The cricket match simulation system is designed as a modular and multithreaded architecture where different components interact with each other to simulate real-time match behavior. Each entity in the system is modeled as an independent unit, and concurrency is achieved through the use of threads.

The overall system is divided into several key modules, each responsible for a specific functionality within the simulation.

1. Batsman Module

The batsman module is responsible for simulating batting actions. It determines the outcome of each ball, such as runs scored or dismissal, based on predefined logic or randomization. Each batsman operates as a thread, allowing independent and concurrent execution.

2. Bowler Module

The bowler module simulates the delivery of the ball. It initiates each play by triggering an event that interacts with the batsman module. The bowler's behavior influences the possible outcomes of each delivery.

3. Fielder Module

The fielder module handles fielding actions such as catching and stopping the ball. It contributes to the realism of the simulation by introducing additional outcomes like run-outs or catches.

4. Scheduler Module

The scheduler acts as the central controller of the simulation. It manages the sequence of events, ensuring proper coordination between the bowler, batsman, and fielders. It also controls the progression of overs and match flow.

5. Scoreboard Module

The scoreboard module maintains all match-related data, including total runs, wickets, overs, and individual player statistics. Since it is a shared resource, it requires synchronization to ensure data consistency.

6. Deadlock Handling Module

This module ensures that the system avoids deadlocks during execution. It implements strategies such as proper lock ordering and controlled resource allocation to maintain smooth operation of threads.

System Workflow

The overall workflow of the system follows a sequential yet concurrent pattern:

- The scheduler initiates the match and controls the sequence of events.
- The bowler delivers the ball.
- The batsman responds by playing a shot.
- The outcome is processed and updated in the scoreboard.
- Fielders may intervene depending on the scenario.
- The cycle repeats for each ball and over until the match ends.

Architecture Overview

The system follows a multithreaded architecture where:

- Each player (batsman, bowler, fielder) is represented as a thread.
- Shared resources such as the scoreboard are protected using synchronization mechanisms.
- The scheduler ensures proper coordination and sequencing of events.

This design ensures efficient utilization of concurrency while maintaining correctness and consistency in the simulation.

IMPLEMENTATION

The cricket match simulation is implemented using a modular approach in C++, where each component of the system is developed as a separate module. The implementation focuses on multithreading, synchronization, and efficient handling of shared resources to simulate real-time match behavior.

1. Thread Creation and Management

Each player entity in the simulation, such as batsmen, bowlers, and fielders, is represented as an independent thread using POSIX threads (pthreads). Threads are created using the pthread_create() function, allowing concurrent execution of different components of the system.

Thread lifecycle management is handled carefully to ensure proper initialization, execution, and termination. Synchronization mechanisms are applied wherever multiple threads interact with shared data.

2. Batsman Implementation

The batsman module is responsible for simulating batting behavior. It processes the delivery from the bowler and determines the outcome of each ball, such as runs scored (0, 1, 2, 4, 6) or dismissal.

The logic is implemented using conditional structures and randomization to mimic realistic gameplay. The batsman thread updates the scoreboard based on the result of each ball.

3. Bowler Implementation

The bowler module initiates each event in the simulation by delivering the ball. It interacts with the batsman thread and triggers the execution of a play.

The bowler's behavior may include variations in delivery, which influence the outcome of the shot. The bowler thread works in coordination with the scheduler to maintain the correct sequence of events.

4. Fielder Implementation

The fielder module handles fielding-related events such as catches and stopping the ball. It introduces additional realism by affecting the outcome of certain plays.

The fielder thread may intervene after the batsman plays a shot, determining whether a catch is taken or a run-out occurs.

5. Scheduler Implementation

The scheduler acts as the control unit of the system. It ensures that all events occur in the correct sequence, such as:

- Ball delivery
- Shot execution
- Score update

It coordinates between different threads and manages the progression of overs and match flow.

6. Scoreboard Management

The scoreboard is implemented as a shared data structure that maintains:

- Total runs
- Wickets
- Overs
- Player statistics

Since multiple threads access and modify the scoreboard, mutex locks are used to ensure safe updates and prevent inconsistencies.

7. Code Structure

The project is organized into multiple source and header files, including:

- batsman.cpp / batsman.h
- bowler.cpp / bowler.h
- fielder.cpp / fielder.h
- scheduler.cpp / scheduler.h
- scoreboard.cpp / scoreboard.h
- deadlock.cpp / deadlock.h

This modular structure improves code readability, maintainability, and scalability.

SYNCHRONIZATION AND DEADLOCK HANDLING

In a multithreaded environment, multiple threads execute concurrently and often share common resources. In this cricket match simulation, shared resources such as the scoreboard and match state variables are accessed by multiple threads, including batsman, bowler, and fielder threads. Without proper control, this can lead to inconsistencies and unpredictable behaviour.

1. Need for Synchronization

Since multiple threads operate simultaneously, there is a possibility of race conditions when two or more threads attempt to access and modify shared data at the same time. For example, if both the batsman and fielder attempt to update the scoreboard concurrently, it may result in incorrect values.

To prevent such issues, synchronization mechanisms are required to ensure that only one thread can access a critical section at a time.

2. Mutex Locks

Mutex (Mutual Exclusion) locks are used to protect shared resources. Before accessing the scoreboard or any shared variable, a thread must acquire the mutex lock using pthread_mutex_lock(). After completing the operation, the thread releases the lock using pthread_mutex_unlock().

This ensures that only one thread modifies the shared data at a time, maintaining data consistency and correctness.

3. Critical Sections

Critical sections are parts of the code where shared resources are accessed or modified. In this project, critical sections include:

- Updating the scoreboard
- Modifying match state variables
- Handling player status (e.g., dismissal)

These sections are protected using mutex locks to avoid concurrent access.

4. Deadlock Conditions

Deadlock is a situation where two or more threads are waiting indefinitely for resources held by each other. This can occur when multiple locks are used without proper ordering.

The necessary conditions for deadlock include:

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

If not handled properly, deadlock can cause the entire simulation to stop.

5. Deadlock Prevention Techniques

To avoid deadlocks in the system, the following strategies are implemented:

- **Lock Ordering:** All threads acquire locks in a predefined and consistent order, preventing circular wait conditions.
- **Minimal Lock Usage:** Locks are held only for the shortest duration necessary to complete critical operations.
- **Avoid Nested Locks:** The system avoids acquiring multiple locks simultaneously wherever possible.
- **Proper Lock Release:** Every acquired lock is released immediately after use to prevent resource blocking.

6. System Stability

By combining synchronization and deadlock prevention techniques, the system ensures:

- Smooth execution of threads
- Consistent and accurate data updates
- No indefinite blocking of threads

This allows the simulation to run efficiently without crashes or unexpected behavior.

ASSUMPTIONS

The cricket match simulation is developed based on a set of assumptions to simplify the implementation and focus on demonstrating core concepts such as multithreading, synchronization, and system design. These assumptions help in reducing complexity while maintaining a realistic representation of match behavior.

- The number of overs in the match is fixed and predefined before the simulation begins.
- Each team consists of a limited number of players, and player roles such as batsman, bowler, and fielder are predefined.
- The outcome of each ball (runs scored, dot ball, wicket, etc.) is determined using predefined logic and randomization rather than real-world physics or player statistics.
- Player performance does not vary dynamically based on skill level; all players are treated uniformly in terms of probability of outcomes.
- The simulation does not include external factors such as weather conditions, pitch variations, or crowd influence.
- Only basic cricket rules are implemented, and advanced rules such as powerplays, no-balls, wides, and field restrictions may be simplified or omitted.
- The scoreboard is assumed to be a centralized shared resource that is accessed by multiple threads in a controlled manner.
- Thread execution timing is dependent on system scheduling and may not perfectly reflect real-time delays.
- Deadlock situations are prevented using predefined strategies, and the system assumes correct implementation of synchronization mechanisms

RESULTS AND OUTPUT

The cricket match simulation was successfully implemented and executed using a multithreaded architecture. The system produces real-time outputs representing the progression of a cricket match, including ball-by-ball events, score updates, and match statistics.

The simulation demonstrates the interaction between multiple threads, such as batsman, bowler, and fielder, while maintaining proper synchronization. The outputs generated reflect realistic match scenarios, including runs scored, wickets taken, and over-by-over progression.

Below are selected outputs from the simulation that illustrate the working and behavior of the system.

Following are the Output in the WSL:

```
anshul@LAPTOP-1A53AGCO:/mnt/c/Users/hp/OneDrive/Desktop/cricket_sim$ make clean
rm -rf obj cricket_sim logs charts
anshul@LAPTOP-1A53AGCO:/mnt/c/Users/hp/OneDrive/Desktop/cricket_sim$ make
mkdir -p obj logs charts
gcc -Wall -Wextra -O2 -g -Iinclude -c src/main.c -o obj/main.o
gcc -Wall -Wextra -O2 -g -Iinclude -c src/init.c -o obj/init.o
gcc -Wall -Wextra -O2 -g -Iinclude -c src/log.c -o obj/log.o
gcc -Wall -Wextra -O2 -g -Iinclude -c src/scheduler.c -o obj/scheduler.o
gcc -Wall -Wextra -O2 -g -Iinclude -c src/pitch.c -o obj/pitch.o
gcc -Wall -Wextra -O2 -g -Iinclude -c src/threads.c -o obj/threads.o
gcc -Wall -Wextra -O2 -g -Iinclude -c src/deadlock.c -o obj/deadlock.o
gcc -Wall -Wextra -O2 -g -Iinclude -c src/analysis.c -o obj/analysis.o
gcc -Wall -Wextra -O2 -g -Iinclude -o cricket_sim obj/main.o obj/init.o obj/log.o obj/scheduler.o obj/pitch.o
obj/threads.o obj/deadlock.o obj/analysis.o -lpthread -lm
Build OK → ./cricket_sim
```

Fig.1 – Making of objects.

```
anshul@LAPTOP-1A53AGCO:/mnt/c/Users/hp/OneDrive/Desktop/cricket_sim$ ./cricket_sim
╔═══════════════════════════════════════════════════╗
║      T20 WORLD CUP 2026 - CRICKET SIMULATOR       ╖
      CSC-204: Operating Systems - Assignment 1
  Scheduling: RR (bowlers) + SJF (batsmen) + Priority
╚═══════════════════════════════════════════════════╝


╔═══════════════════════════════════════════════╗
║              INNINGS 1 BEGINS                 ╖
╚═══════════════════════════════════════════════╝

=== INNINGS 1 BEGINS ===
  Striker     : Rohit Sharma
  Non-Striker : Virat Kohli
  Bowler (RR) : Jasprit Bumrah

[CREASE] Rohit Sharma entered crease (sem acquired)
  Over  0.1 | Jasprit Bumrah       → Rohit Sharma         |   2 | 2/0
[CREASE] Virat Kohli entered crease (sem acquired)
  Over  0.2 | Jasprit Bumrah       → Rohit Sharma         |   2 | 4/0
[MAIN] 27 threads spawned (6 bowlers + 11 batsmen + 10 fielders)

  Over  0.3 | Jasprit Bumrah       → Virat Kohli          |   1 | 5/0
  Over  0.3 | Jasprit Bumrah       → Rohit Sharma         |  WD | 6/0
  Over  0.4 | Jasprit Bumrah       → Rohit Sharma         | DOT | 6/0
  Over  0.5 | Jasprit Bumrah       → Rohit Sharma         |   2 | 8/0
```

Fig.2 – Innings 1 (Ind Vs. Aus)

T20 WORLD CUP 2026 — INNINGS 1 SCORECARD

BATTING

Batsman	Runs	Balls	SR	
Rohit Sharma	21	8	262.5	out
Virat Kohli	0	1	0.0	out
Arshdeep Singh	0	1	0.0	out
Jasprit Bumrah	0	2	0.0	out
Kuldeep Yadav	84	46	182.6	out
Axar Patel	13	11	118.2	out
Dinesh Karthik	6	3	200.0	out
Rinku Singh	55	30	183.3	out
Hardik Pandya	24	17	141.2	dnb
Suryakumar Yadav	3	1	300.0	dnb

TOTAL: 206/8 (Overs 20.0)

BOWLING

Bowler	Ovr	Runs	Wkts	Econ
Jasprit Bumrah	4	40	4	10.00 ★
Mohammed Siraj	4	55	1	13.75
Hardik Pandya	3	26	0	8.67
Axar Patel	3	26	1	8.67
Kuldeep Yadav	2	23	0	11.50
Arshdeep Singh	4	36	1	9.00 ★

(★ = Death Over Specialist — Priority Scheduled)

Fig.3 –Scorecard of Innings 1 (Ind Vs. Aus)

T20 WORLD CUP 2026 — INNINGS 2 SCORECARD

BATTING

Batsman	Runs	Balls	SR	
David Warner	7	8	87.5	out
Travis Head	7	3	233.3	out
Josh Hazlewood	31	21	147.6	out
Adam Zampa	95	54	175.9	out
Mitchell Starc	20	12	166.7	out
Pat Cummins	2	3	66.7	out
Matthew Wade	25	8	312.5	out
Tim David	17	7	242.9	out
Glenn Maxwell	7	2	350.0	dnb
Steven Smith	5	2	250.0	dnb

TOTAL: 216/8 (Overs 20.0)

BOWLING

Bowler	Ovr	Runs	Wkts	Econ
Jasprit Bumrah	4	37	2	9.25 ★
Arshdeep Singh	4	42	1	10.50 ★
Hardik Pandya	3	27	3	9.00
Axar Patel	3	31	0	10.33
Kuldeep Yadav	3	24	1	8.00
Mohammed Siraj	3	55	1	18.33

(★ = Death Over Specialist — Priority Scheduled)

Fig.4 – Scorecard of Innings 2 (Ind Vs. Aus)

ANALYSIS

The cricket match simulation demonstrates the effective application of multithreading, scheduling algorithms, and synchronization techniques in a dynamic environment. The results obtained from execution logs and graphical visualizations provide insights into system behavior, performance, and correctness.

1. Scheduling Algorithm Behavior

The simulation incorporates multiple scheduling strategies, including Round Robin (RR), Shortest Job First (SJF), and Priority-based scheduling.

From the execution logs (Innings 1 and Innings 2), it is observed that:

- **Round Robin scheduling** ensures fair distribution of overs among bowlers, maintaining balance in workload.
- **SJF scheduling** prioritizes batsmen with fewer expected balls, resulting in significantly lower waiting times compared to FCFS.
- **Priority scheduling** activates death-over specialists in high-intensity phases (after specific overs), improving match realism.

The comparison between **SJF and FCFS waiting times** clearly shows that SJF reduces waiting time for middle-order batsmen, improving overall efficiency.

```
╔═══ INNINGS 1 — SCHEDULING ANALYSIS (All 3 Algorithms) ═══╗

A) ROUND-ROBIN (Bowler Over Rotation)
   Quantum = 6 balls. After each over, context switch.
   Max overs per bowler = 4 (T20 rule enforced).

B) SJF (Incoming Batsman Selection on Wicket)
   Batsman                 Exp.Balls    SJF-Wait   FCFS-Wait
   ──────────────────────────────────────────────────────────
   Jasprit Bumrah                3           1          10
   Kuldeep Yadav                 5           2          12
   Axar Patel                    7           2          58
   Dinesh Karthik               10           4          69
   Rinku Singh                  14           5          72
   Hardik Pandya                18           7         102

   SJF promotes tail-enders (shorter burst) ahead of
   middle-order → lower average wait for middle-order.

C) PRIORITY (Death Over Specialist)
   High Intensity triggered at over >= 14
   Death specialists used: Jasprit Bumrah  Arshdeep Singh
   Match_Intensity was HIGH (death specialists activated) during this simulation.

Match Summary
   Total Balls : 120
   Total Runs  : 206
   Wickets     : 8
   Overs       : 20.0
```

Fig.5 – Analysis of Innings 1

2. Concurrency and Thread Execution

The system models each player as an independent thread, leading to concurrent execution of match events.

From the logs:

- Events such as ball delivery, shot execution, and score updates occur in rapid succession.
- The sequence of events varies across runs, demonstrating **non-deterministic thread scheduling**, a key characteristic of multithreaded systems.

Despite concurrency, the system maintains correct outputs, indicating proper synchronization.

```
╔═══ INNINGS 2 — SCHEDULING ANALYSIS (All 3 Algorithms) ═══╗

A) ROUND-ROBIN (Bowler Over Rotation)
   Quantum = 6 balls. After each over, context switch.
   Max overs per bowler = 4 (T20 rule enforced).

B) SJF (Incoming Batsman Selection on Wicket)
   Batsman                 Exp.Balls    SJF-Wait   FCFS-Wait
   ──────────────────────────────────────────────────────────
   Adam Zampa                     3           1          32
   Mitchell Starc                 5           2          86
   Pat Cummins                    7           2          98
   Matthew Wade                  10           4         101
   Tim David                     15           6         109
   Glenn Maxwell                 20           8         116

   SJF promotes tail-enders (shorter burst) ahead of
   middle-order → lower average wait for middle-order.

C) PRIORITY (Death Over Specialist)
   High Intensity triggered at over >= 14
   Death specialists used: Jasprit Bumrah  Arshdeep Singh
   Match_Intensity was HIGH (death specialists activated) during this simulation.

Match Summary
   Total Balls : 120
   Total Runs  : 216
   Wickets     : 8
   Overs       : 20.0
```

Fig.6 – Analysis of Innings 2

3. Synchronization Effectiveness

The scoreboard and match state are shared resources accessed by multiple threads.

From both logs and score progression graphs:

- No inconsistencies in runs, wickets, or overs are observed.
- Score updates follow a consistent and valid progression.

This confirms that:

- Mutex locks successfully prevent race conditions
- Critical sections are properly protected

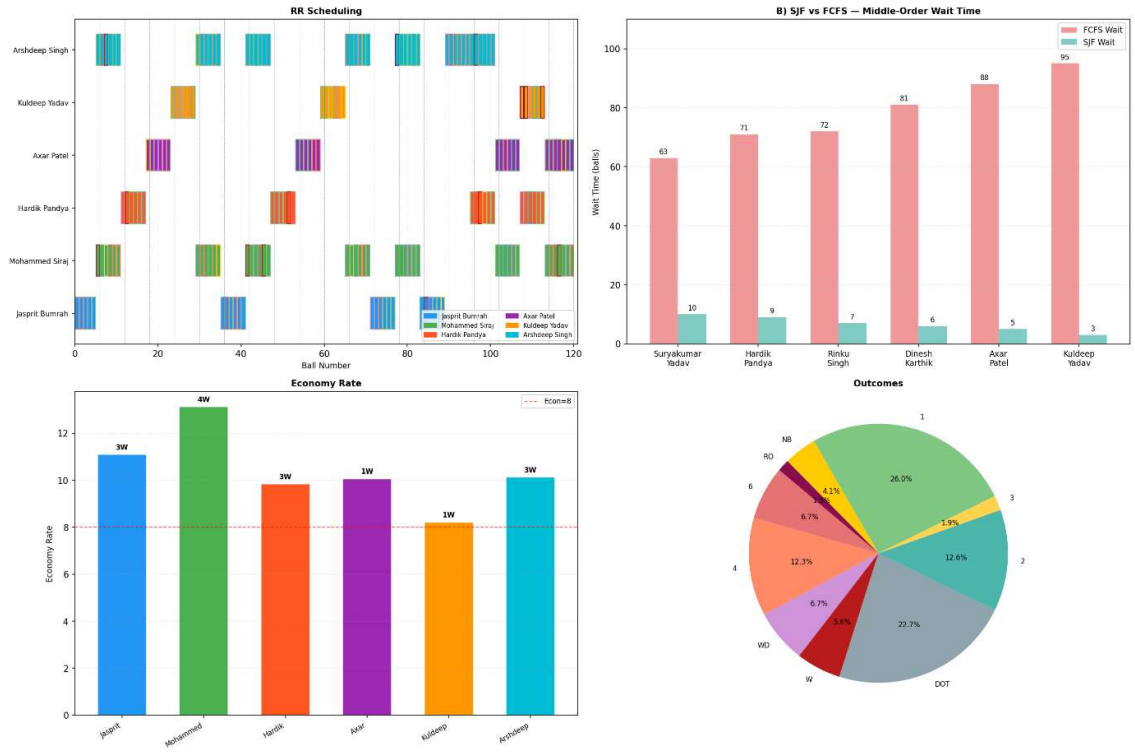

Fig.7 – Scheduling & Performance Visualizations

4. Performance Analysis Using Visualizations

The graphical outputs provide deeper insights into system performance:

- The **Gantt chart of bowlers (Round Robin)** shows balanced distribution of overs, validating fairness of scheduling.
- The **score timeline graph** shows steady growth of runs with discrete jumps at scoring events and wickets.
- The **economy rate chart** highlights variation in bowler performance, indicating realistic simulation behavior.
- The **outcome distribution (pie chart)** shows frequency of events such as runs, dot balls, wides, and wickets, reflecting probabilistic modeling.

These visualizations confirm that the simulation produces structured and interpretable results.

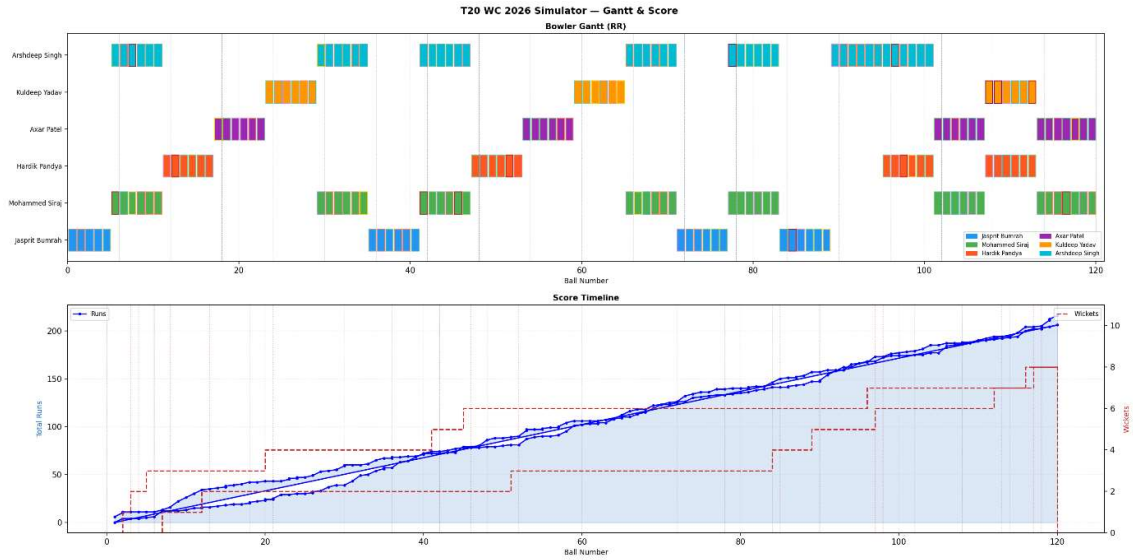

Fig.8 --Gantt Chart and Score Timeline

5. System Stability and Correctness

The simulation executes completely without:

- Deadlocks
- Crashes
- Inconsistent outputs

The logs show full match completion (20 overs), confirming:

- Proper thread termination
- Correct synchronization
- Stable system behavior

6. Realism and Practical Insights

The integration of scheduling algorithms with cricket simulation provides a unique perspective:

- Scheduling concepts map effectively to real-world cricket strategies
- Death-over prioritization reflects real match tactics
- Performance metrics (runs, economy, outcomes) enhance realism

Although simplified, the system successfully captures key aspects of a real cricket match.

CONCLUSION

The multithreaded cricket match simulation successfully demonstrates the application of core operating system concepts in a practical and engaging scenario. By modeling players such as batsmen, bowlers, and fielders as independent threads, the system effectively simulates concurrent execution similar to real-world match dynamics.

The project highlights the importance of synchronization mechanisms, particularly mutex locks, in ensuring safe access to shared resources like the scoreboard. Proper handling of critical sections prevents race conditions and maintains data consistency throughout the simulation. Additionally, the implementation incorporates strategies to prevent deadlocks, ensuring smooth and uninterrupted execution of the system.

The modular design of the system improves readability, maintainability, and scalability, while the use of POSIX threads (pthreads) enables efficient thread management. The simulation produces realistic outputs, including ball-by-ball events, score progression, and match results, providing a clear demonstration of system behavior.

Overall, this project bridges the gap between theoretical concepts and practical implementation, offering valuable insights into multithreading, synchronization, and system design. It serves as an effective example of how operating system principles can be applied to model complex rea